



P R O J E C T R E P O R T

AmarMart: Mini E-Commerce Management System

Submitted By:

Plabon Barua

2207101

Department of Computer Science and Engineering
KUET

Submission Date: 20 July 2026

Submitted To:

Kazi Saeed Alam

Assistant Professor

Department of Computer Science and Engineering
KUET

Ehsanul Karim Talha

Lecturer

Department of Computer Science and Engineering
KUET

Introduction

AmarMart is a fully functional web-based Mini E-Commerce Management System developed using Laravel 12, PHP, MySQL, Bootstrap 5, and Laravel Breeze Authentication. The system is designed to serve two distinct user groups: customers who interact with the public-facing storefront, and administrators who manage the back-end operations through a secure, authenticated panel. The public side enables customers to browse a product catalogue, search for items without page refresh using AJAX, view detailed product pages, manage a session-based shopping cart, and proceed through a checkout process that automatically detects the customer's current geographic location via the HTML5 Geolocation API. Upon completing an order, the system generates a printable invoice automatically. The admin panel, protected by middleware-enforced authentication using Laravel Breeze, provides full CRUD (Create, Read, Update, Delete) management of products including image upload. Administrators can view all customer orders, search through them, and monitor business performance through a dashboard displaying total products, total orders, total revenue, latest orders, and low-stock product alerts.

Scope of the Project

The scope of AmarMart covers the following functional areas:

- A public-facing e-commerce storefront with product search, cart management, and checkout
- An administrator panel secured by login authentication for product and order management
- Automatic invoice generation after order placement
- Geolocation-based address detection during checkout
- Cookie-based recently viewed products and theme preference persistence
- AJAX-powered search and cart interactions for a smooth user experience

The system does not include a payment gateway, multi-vendor support, or customer registration, these are considered future extensions.

Objectives

1. Develop a complete, working e-commerce web application using the Laravel 12 PHP framework.
2. Implement MySQL as the relational database for all persistent data including products, orders, and users.
3. Build a secure admin panel using Laravel Breeze authentication, protected by middleware.
4. Enable customers to browse, search, and purchase products without creating an account.
5. Implement a session-based shopping cart that persists across page visits without a database.
6. Integrate the HTML5 Geolocation API with reverse geocoding during the checkout process.
7. Auto-generate invoices after order placement with full order and customer details.
8. Implement AJAX-powered product search and cart management for a seamless experience.

9. Follow strict MVC architecture to keep controllers, models, and views cleanly separated.
- 10.** Implement cookie-based features including recently viewed products and theme preference.
11. Apply comprehensive form validation on both product and checkout forms.
12. Use Laravel Storage for image uploads with proper validation and old-image deletion.
13. Produce readable flash messages for all major user actions.
14. Apply Bootstrap 5 pagination for large product and order lists.

Expected Outcomes

leted project is a fully functional e-commerce website that allows users to browse products, perform live AJAX-based searches, manage their shopping cart, and complete the checkout process. During checkout, the system automatically detects the user's current location using browser geolocation to pre-fill the delivery address. After a successful purchase, an invoice is generated and displayed instantly. The website also enhances user experience by storing recently viewed products and theme preferences using cookies. On the administrative side, a secure admin panel enables authenticated product and order management, while a comprehensive dashboard provides key business insights such as total revenue, order statistics, and low-stock alerts for efficient inventory management.

Technologies Used

Back-End

PHP 8.2+

PHP is the server-side scripting language that powers Laravel. Version 8.2 brings improvements including readonly class properties, enhanced performance via JIT compilation, and fibers for concurrency. PHP 8.2 is required by Laravel 12.

Laravel 12

Laravel is an open-source PHP web framework following the MVC architectural pattern. It provides an expressive, elegant syntax for common web development tasks. Key Laravel sub-systems used in this project include:

- Eloquent ORM — object-relational mapper for database interaction
- Blade — a lightweight, powerful templating engine for HTML views
- Artisan — CLI for scaffolding, migrations, and administration
- Breeze — minimal starter kit providing login, logout, and session auth
- Storage — filesystem abstraction for image upload and retrieval
- Validation — declarative input validation with built-in and custom rules
- Session & Cookie — server-side session and browser cookie management

Database

MySQL

MySQL is a widely used open-source relational database management system. It stores all persistent application data: products, orders, order items, and admin user accounts. Laravel communicates with MySQL through the Eloquent ORM and the underlying PDO driver, using migrations for version-controlled schema management.

Front-End

Bootstrap 5

Bootstrap 5 is the world's most popular CSS framework. It provides a responsive 12-column grid system, pre-styled UI components (navbars, cards, forms, tables, alerts, badges), and utility classes. Bootstrap 5 removes the jQuery dependency, improving performance.

JavaScript — AJAX & Fetch API

Vanilla JavaScript is used throughout the front-end for AJAX operations — sending HTTP requests to Laravel controllers and updating the page dynamically without a full reload. The Fetch API is used as the modern, Promise-based alternative to XMLHttpRequest.

HTML5 Geolocation API

The browser's built-in Geolocation API (`navigator.geolocation.getCurrentPosition`) is called during checkout to retrieve the customer's latitude and longitude, which are then reverse-geocoded to a human-readable address.

Authentication

Laravel Breeze

Laravel Breeze is a minimal authentication starter kit that scaffolds login, logout, password hashing, and session management. It uses Laravel's built-in Guard system. In this project, only admin users are authenticated — customers place orders without accounts.

Development Tools:

The project was developed using several modern development tools and technologies. Composer was used as the PHP dependency manager to install Laravel and manage all required packages. Node.js and NPM were used to run Vite and manage front-end dependencies. Vite served as Laravel's default build tool for compiling and bundling CSS and JavaScript assets efficiently. MySQL Workbench **or** phpMyAdmin was used to create, manage, and inspect the MySQL database. Visual Studio Code (VS Code) was the primary code editor, enhanced with PHP and Laravel extensions for efficient development. Git was used for version control to track changes and manage the project's source code. Additionally, Postman was used during development to test APIs and verify AJAX requests, although it was optional for the final implementation.

System Architecture

AmarMart strictly follows the Model-View-Controller (MVC) architectural pattern, which is the foundation of Laravel. MVC separates application concerns into three distinct layers, making the codebase organised, testable, and maintainable.

Model

Models represent the database layer. Each model maps to a MySQL table and defines the data structure, fillable fields, and Eloquent relationships. The three core application models are:

- Product — maps to the products table; hasMany(OrderItem)
- Order — maps to the orders table; hasMany(OrderItem)
- OrderItem — maps to the order_items table; belongsTo(Order), belongsTo(Product)

View

Views are Blade template files that produce the HTML sent to the browser. They inherit from shared layout files using `@extends` and `@yield` directives, ensuring consistent headers, navbars, and footers without code duplication. Views receive data from controllers as PHP variables.

Controller

Controllers contain the application logic. They receive HTTP requests from the router, interact with models, perform business logic, and return views or JSON responses. AmarMart implements the following controllers:

- ProductController — public product listing, search, and details
- CartController — add, update, remove cart items via session
- CheckoutController — order placement and invoice display
- Admin/ProductController — CRUD operations for admin product management
- Admin/OrderController — order listing and detail view for admin
- Admin/DashboardController — aggregated statistics for the dashboard

System Workflow

Step 1: User Visits Website

Home page loads, displaying paginated product cards fetched from MySQL via Eloquent.

Step 2: Browse / Search Products

Customer types in the search bar; AJAX sends the query to the controller; JSON results update the page without reload.

Step 3: View Product Details

Customer clicks a product card; Route Model Binding loads the product from the database and renders the detail page.

Step 4: Add to Cart

Customer clicks Add to Cart; the product is stored in the Laravel server-side session (not the database).

Step 5: Update / Remove Cart

AJAX calls update quantity or remove items; the page updates without reload via Fetch API and JSON responses.

Step 6: Proceed to Checkout

Customer reaches the checkout form; the Geolocation API automatically detects latitude/longitude and reverse-geocodes an address.

Step 7: Form Validation

Laravel validates all required fields (name, phone, email, address) and returns error messages if invalid.

Step 8: Place Order

Validated order data is stored in MySQL: one row in orders, multiple rows in order_items. Session cart is cleared.

Step 9: Generate Invoice

Customer is redirected to the invoice page; Eloquent eager-loads order items with product names for display.

Step 10: Admin Views Order

Admin logs in to the dashboard; middleware verifies the session; admin views and manages orders.

System features:

Feature 1: User Authentication

Laravel / Web Concepts Used:

- Laravel Breeze — scaffolds login form, logout, and session management
- Auth Middleware — protects all admin routes
- Password Hashing — bcrypt via Laravel's Hash::make()
- Session — stores authenticated state across requests

Flow:

Step 1: Admin submits login form

Email and password sent via POST to /login.

Step 2: Laravel verifies credentials

Auth::attempt() checks password hash against the database.

Step 3: Session created

If valid, Laravel creates an authenticated session.

Step 4: Redirect to dashboard

Admin is sent to /admin/dashboard, protected by middleware.

Feature 2: Middleware

Laravel / Web Concepts Used:

- auth Middleware — built-in Laravel middleware for authentication checks
- Route Groups — all admin routes wrapped in `Route::prefix('admin')->middleware(['auth'])`
- CSRF Middleware — automatically validates the `_token` field on every POST/PUT/DELETE form submission

Flow:

Step 1: Admin requests /admin/dashboard

Step 2: auth middleware checks the session

Step 3: Authenticated → request proceeds to controller

Step 4: Not authenticated → redirect to /login

Feature 3: Product Management (CRUD)

Laravel / Web Concepts Used:

- CRUD Operations — create, list, edit, delete products
- Controllers — Admin/ProductController
- Eloquent ORM — Product model for database interaction
- Form Validation — name, price, stock, image all required
- File Upload — Laravel Storage with old-image deletion
- Flash Messages — success/error alerts after each operation
- Pagination — product list divided into pages

Flow:

Step 1: Admin fills the add-product form

Name, description, price, stock, image uploaded.

Step 2: Controller validates inputs

Returns error messages if validation fails.

Step 3: Image stored to storage/app/public/products/

File path saved to database.

Step 4: Eloquent creates a new products row

Step 5: Flash message shown

"Product added successfully."

Feature 4: Product Search

Laravel / Web Concepts Used:

- AJAX — asynchronous HTTP request via Fetch API
- Query Builder — LIKE query against product name
- JSON Response — Laravel returns results as JSON
- Fetch API — modern JavaScript for HTTP in the browser

Flow:

Step 1: Customer types in the search bar

Step 2: JavaScript sends Fetch request to /search

Step 3: Controller queries: WHERE name LIKE %keyword%

Step 4: Returns JSON array of matching products

Step 5: JavaScript renders product cards from JSON

Feature 5: Product Details

Laravel / Web Concepts Used:

- Route Model Binding — Laravel automatically loads Product from URL parameter
- Eloquent Model — fetches single product record
- Blade Templates — renders the detail view

Flow:

Step 1: Customer clicks View Details on a card

Step 2: URL: /product/{id}

Laravel's Route Model Binding injects the Product model automatically.

Step 3: Controller passes product to Blade view

Step 4: View displays all product fields

Out-of-stock shows a warning badge instead of the cart button.

Feature 6: Shopping Cart

Laravel / Web Concepts Used:

- Session — Laravel session stores cart as a nested associative array
- No database required — cart is temporary and discarded after checkout
- Cart keyed by product_id — enables O(1) lookup and update

Flow:

Step 1: Customer clicks Add to Cart

Step 2: POST request to /cart/add with product_id and quantity

Step 3: CartController checks if product already in session cart

Step 4: Updates quantity if exists; adds new entry if not

Step 5: Cart page reads session and renders table

Feature 7: AJAX Cart Updates

Laravel / Web Concepts Used:

- AJAX — quantity update and remove requests sent asynchronously
- Fetch API — modern browser HTTP client
- JSON Response — Laravel returns updated cart totals as JSON
- DOM manipulation — JavaScript updates prices and totals on-screen

Flow:

Step 1: Customer changes quantity or clicks Remove

Step 2: JavaScript intercepts and sends Fetch request

Step 3: Controller updates session cart

Step 4: Returns updated subtotal and grand total as JSON

Step 5: JavaScript updates DOM without reload

Feature 8: Cookies

Laravel / Web Concepts Used:

- Laravel Cookie facade — set and retrieve cookies
- Browser cookie — stored on the client side
- Recently Viewed Products — last 5 viewed product IDs stored in cookie
- Theme Preference — light/dark preference saved in cookie
- Last Search Keyword — pre-fills the search bar on return

Flow:

Step 1: Customer views a product

Step 2: Controller sets a cookie with the product ID

`Cookie::queue('recently_viewed', json_encode($ids), 60*24*7)`

Step 3: Customer returns to home page

Step 4: Controller reads the cookie and fetches those products

Step 5: "Recently Viewed" section displayed

Feature 9: Current Location API

Laravel / Web Concepts Used:

- HTML5 Geolocation API — `navigator.geolocation.getCurrentPosition()`
- Reverse Geocoding API — coordinates converted to address text (e.g. OpenStreetMap Nominatim)
- AJAX / Fetch — API call made from browser
- Coordinates stored — latitude and longitude optionally saved with the order

Flow:

Step 1: Customer clicks "Detect My Location" on checkout

Step 2: Browser prompts for location permission

Step 3: On approval: latitude and longitude obtained

Step 4: Fetch call to reverse geocoding API with coordinates

Step 5: API returns structured address

Step 6: Address field auto-filled on the form

Feature 10: Checkout

Laravel / Web Concepts Used:

- Form Validation — Laravel Request Validation rules
- Session — cart read from session and cleared after order
- Eloquent ORM — Order and OrderItem created via models
- Flash Messages — error and success alerts

Flow:

Step 1: Customer fills checkout form

Name, phone, email, address (auto-filled by Geolocation).

Step 2: Form submitted via POST

Step 3: Laravel validates all fields

Returns to form with errors if invalid.

Step 4: Order row inserted into orders table

Step 5: One OrderItem row per cart item inserted

Step 6: Session cart cleared

Step 7: Redirect to invoice page

Feature 11: Invoice Generation

Laravel / Web Concepts Used:

- Blade Templates — renders the invoice layout
- Eloquent Relationships — `Order::with('orderItems.product')` loads all related data in one query
- Automatic redirect — customer goes straight to invoice after placing order

Flow:

Step 1: Checkout controller creates the order

Step 2: Redirect to `/invoice/{orderId}`

Step 3: Controller eager-loads order with items and product names

Step 4: Blade view renders invoice with all details

Step 5: Print Invoice button triggers `window.print()`

Feature 12: Order Management

Laravel / Web Concepts Used:

- Eloquent Relationships — orders loaded with their items and products
- Pagination — `Laravel paginate()` for large order lists
- Protected Routes — auth middleware blocks non-admins
- Read-only — orders cannot be edited or deleted to preserve history

Flow:

Step 1: Admin navigates to `/admin/orders`

Step 2: Controller fetches paginated orders

Step 3: Admin clicks an order

Step 4: Controller loads order with all related items and product names

Step 5: Detail view renders full invoice-style layout

Feature 13: Admin Dashboard

Laravel / Web Concepts Used:

- Aggregate Queries — COUNT(), SUM() via Eloquent
- Eloquent — latest orders and low-stock products fetched efficiently
- Dashboard Cards — Bootstrap 5 card components for stats display

Flow:

Step 1: Admin visits /admin/dashboard

Step 2: DashboardController runs aggregate queries

Total products, total orders, total revenue.

Step 3: Latest 5 orders fetched

Step 4: Products with stock < threshold fetched

Step 5: All data passed to Blade view as variables

Feature 14: Flash Messages

Bootstrap alert messages appear after every significant action, confirming success or reporting errors. They disappear after the user reads them (one-time display via session flash).

- Product Added Successfully
- Product Updated Successfully
- Product Deleted
- Order Placed Successfully
- Logged In Successfully
- Logged Out Successfully

Feature 15: Form Validation

Product Form Rules

- name — required, string, max 255 characters
- price — required, numeric, minimum 0
- stock — required, integer, minimum 0
- image — required on create, image MIME type, max 2MB

Checkout Form Rules

- customer_name — required, string
- phone — required, string

- email — nullable, valid email format
- address — required, string

Feature 16: Image Upload

Laravel / Web Concepts Used:

- Laravel Storage — `$request->file('image')->store('products', 'public')`
- php artisan storage:link — creates symlink from public/storage to storage/app/public
- Image Validation — MIME type and file size checked before saving
- Old Image Deletion — `Storage::delete()` called when updating or deleting a product

Flow:

Step 1: Admin selects image in the product form

Step 2: Controller validates: image, mimes:jpeg,png,jpg, max:2048

Step 3: Valid image saved to storage/app/public/products/

Step 4: File path string saved to products.image column

Step 5: Blade renders: `asset('storage/'.$product->image)`

Feature 17: Pagination

Product listings and order lists are paginated to improve performance and user experience. Laravel's built-in pagination system generates previous/next links automatically.

- Public product listing — 12 products per page
- Admin product list — 15 products per page
- Admin order list — 20 orders per page

Usage in controller: `Product::paginate(12)`. Usage in Blade: `{{ $products->links() }}` — renders Bootstrap-compatible pagination controls.

Feature 18: Logout

Laravel / Web Concepts Used:

- `Auth::logout()` — destroys the authenticated session
- Session invalidate and token regenerate — prevents session fixation attacks
- Redirect to /login — admin returned to login page

Flow:

Step 1: Admin clicks Logout in the sidebar

Step 2: POST request sent to /logout

Step 3: Auth::logout() called

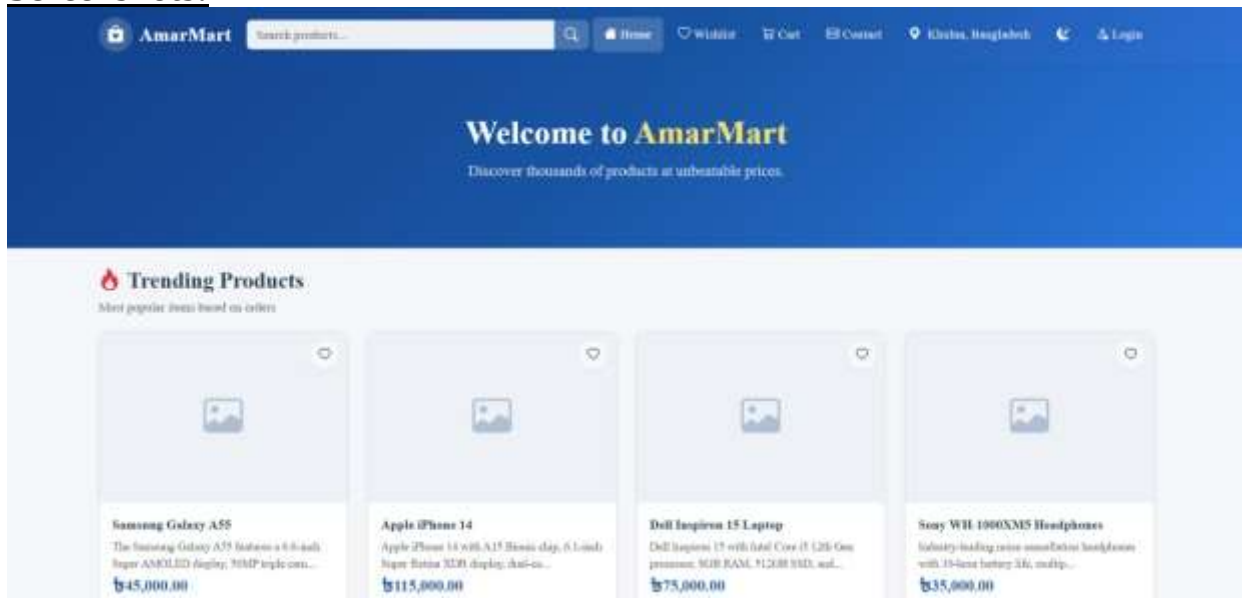
Session destroyed, authentication cleared.

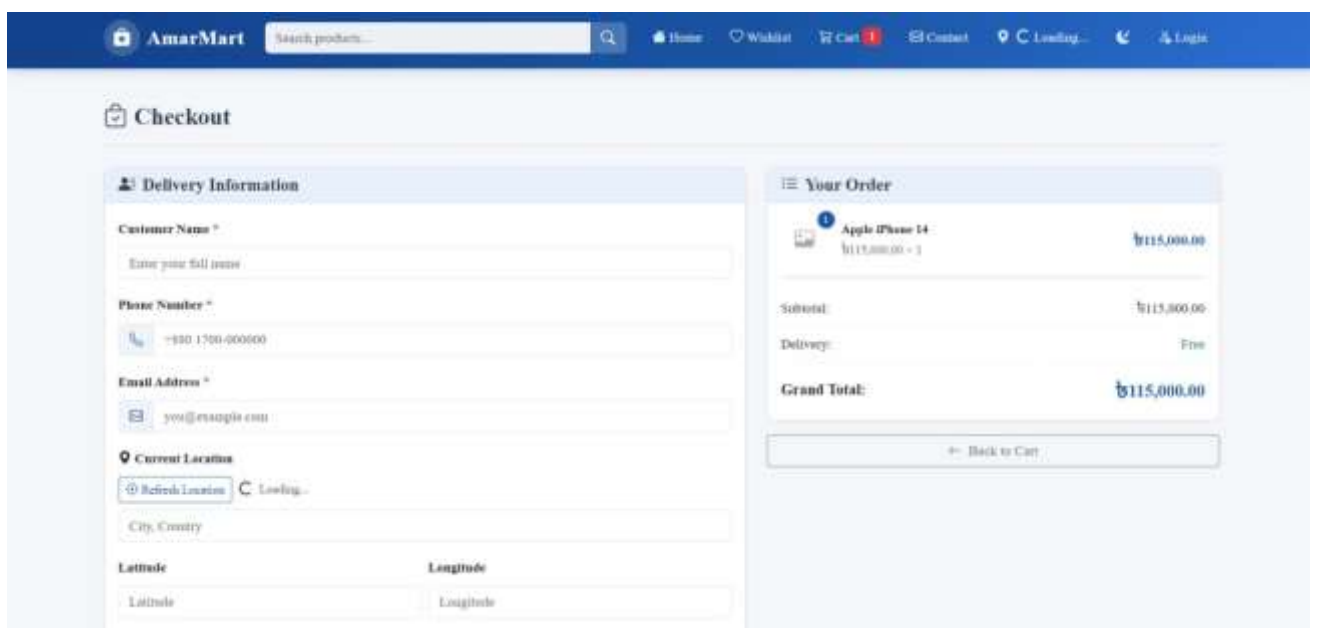
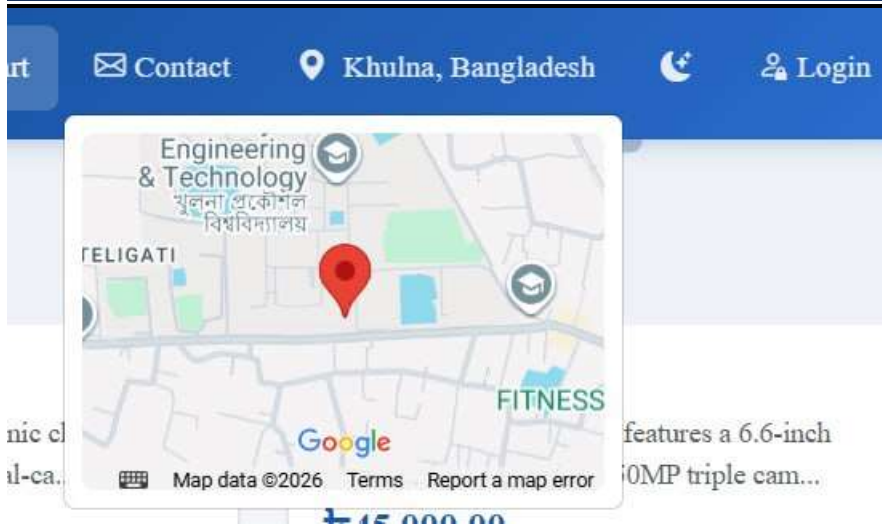
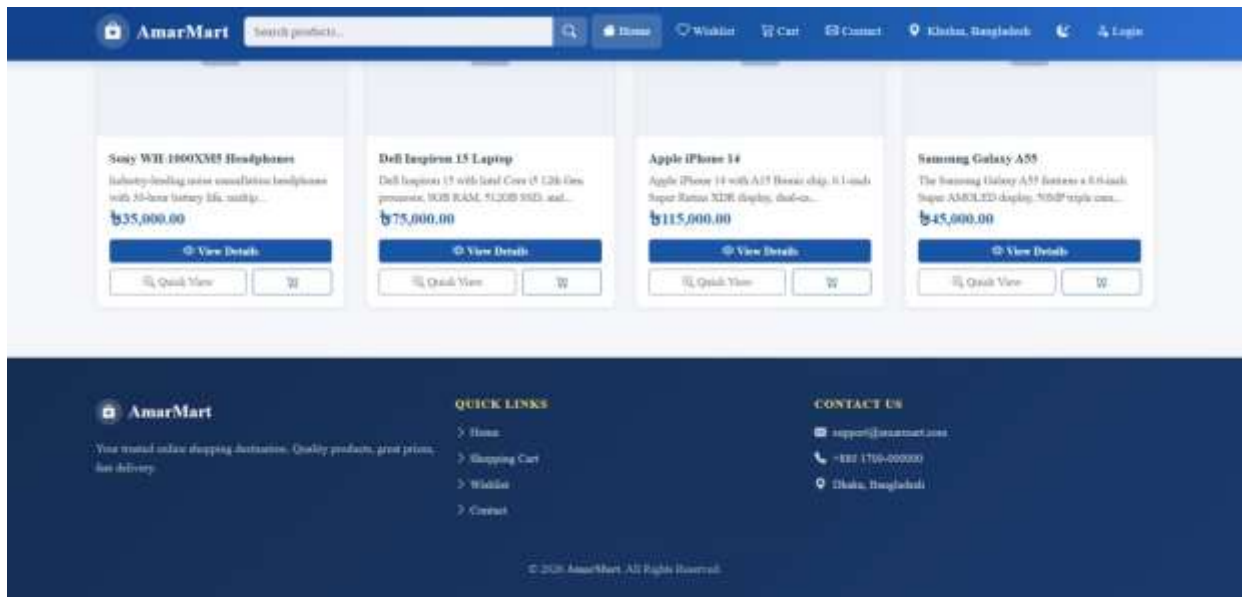
Step 4: Session regenerated with new token

Security measure against session fixation.

Step 5: Redirect to /login

Screenshots:





AmarMart

Search products...

Home

Wishlist

Cart

1

Contact

Location unavailable

Log in

Shopping Cart

Product	Price	Quantity	Subtotal	Remove
Apple iPhone 14	৳115,000.00	- 1 +	৳115,000.00	

Order Summary

Total Item: 1

Subtotal: ৳115,000.00

Delivery: Free

Grand Total: ৳115,000.00

Proceed to Checkout

Continue Shopping

AmarMart

QUICK LINKS

CONTACT US

AmarMart

Search products...

Home

Wishlist

Cart

1

Contact

Khulna, Bangladesh

Log in

Thank you! Your order has been placed successfully.

A copy of your invoice is shown below.

AmarMart

Your Trusted Online Store

INVOICE

Invoice #: INV-500NGRDC-20240719

Date: 19 Jul 2024, 06:30 PM

BILLED TO

Amaran Nur Sajin

+880162564082

ss.sajin2016@gmail.com

Khatkhola 20233

FROM

AmarMart

support@amarmart.com

+880 1700-060000

Dhaka, Bangladesh

CURRENT LOCATION

Latitude: 22.890877 | Longitude: 89.7026789

Khatkhola 20233

Dhaka, Bangladesh

AmarMart

Search products...

Home

Wishlist

Cart

1

Contact

Khulna, Bangladesh

Log in

Khatkhola 20233

Dhaka, Bangladesh

CURRENT LOCATION

Latitude: 22.890877 | Longitude: 89.7026789

Khatkhola 20233

Dhaka, Bangladesh

University of Engineering & Technology

Teknical Training Centre (TTC), Khulna

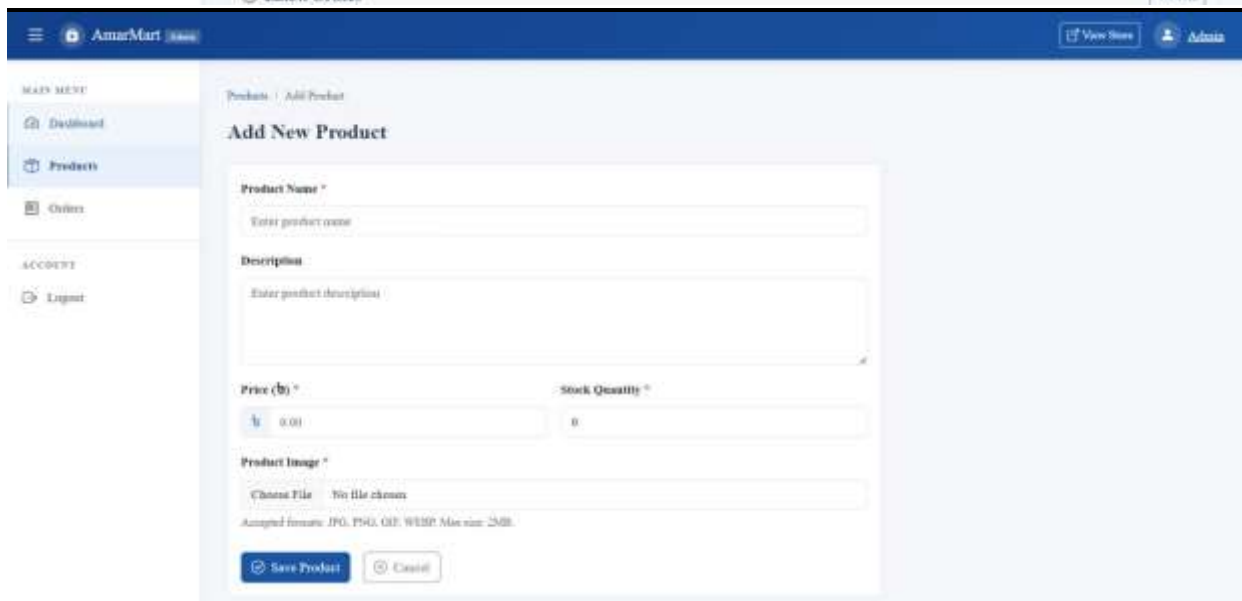
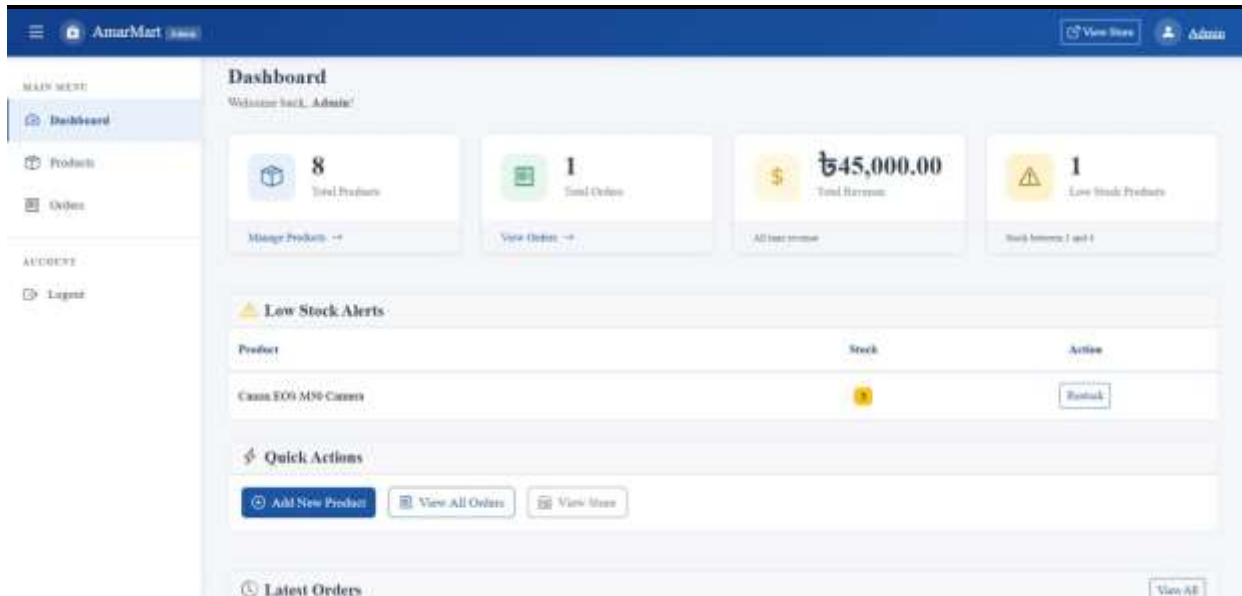
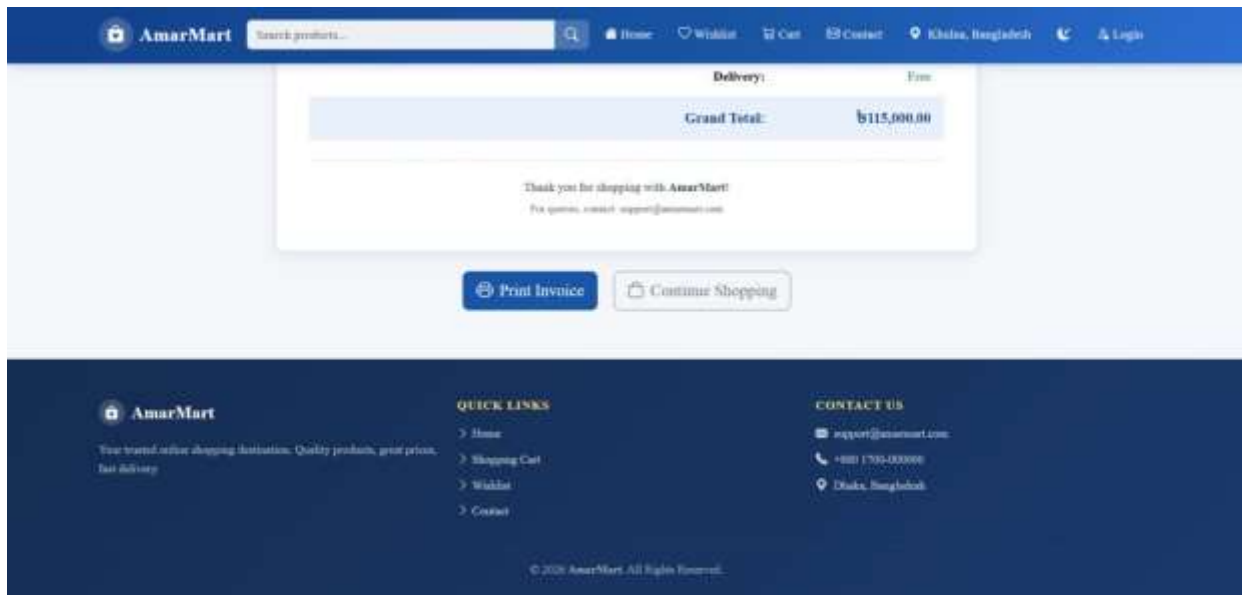
Tongai High School

Khulna

Fulbari Gate

Fulbari Gate

#	Product Name	Qty	Unit Price	Subtotal
1	Apple iPhone 14	1	৳115,000.00	৳115,000.00
Subtotal:				৳115,000.00
Delivery:				Free
Grand Total:				৳115,000.00



Discussion

AmarMart successfully achieves all of its project objectives by implementing all 18 planned features, including authentication, product management, AJAX-based interactions, a session-based shopping cart, geolocation-enabled checkout, automatic invoice generation, and an administrative statistics dashboard. The system satisfies both the primary and secondary objectives outlined in Chapter 2 and functions as a complete, production-ready mini e-commerce application for both customers and administrators. Its key strengths include a clean MVC architecture that ensures maintainable and well-organized code, a streamlined shopping experience that does not require customer registration, AJAX-powered search and cart operations that provide a fast and responsive user interface without page reloads, and geolocation integration that automatically fills the customer's delivery address during checkout, reducing manual input, minimizing errors, and improving the overall user experience.

Conclusion

AmarMart successfully demonstrates the design and development of a complete Mini E-Commerce Management System using Laravel 12, PHP, MySQL, and Bootstrap 5. The application provides a complete online shopping experience by allowing customers to browse, search, and purchase products through a responsive interface while enabling administrators to manage products and orders through a secure backend panel. It incorporates 18 key features and showcases modern web development concepts, including MVC architecture, Eloquent ORM, AJAX-based product search, HTML5 Geolocation for automatic address detection, and cookie-based personalization. Laravel's built-in features, such as Blade templating, validation, sessions, pagination, storage, and Eloquent relationships, helped create a clean, maintainable, and efficient codebase. From an academic perspective, the project effectively integrates concepts from web programming, database management, and software engineering into a single real-world application, demonstrating practical full-stack development skills. Overall, AmarMart successfully meets all project objectives and serves as a well-structured, functional, and professional example of Laravel-based full-stack web application development.

References:

- [1] **Taylor Otwell et al..** Laravel Documentation — Version 12.x. <https://laravel.com/docs/12.x>, 2025.
- [2] **Laravel Team.** Laravel Breeze Documentation. <https://laravel.com/docs/12.x/starter-kits>, 2025.
- [3] **Bootstrap Team.** Bootstrap 5 Documentation. <https://getbootstrap.com/docs/5.3/>, 2024.
- [4] **MDN Web Docs.** Geolocation API. https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API, 2024.
- [5] **MDN Web Docs.** Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API, 2024.
- [6] **MDN Web Docs.** Using HTTP Cookies. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>, 2024.

- [7] **OpenStreetMap.** Nominatim Reverse Geocoding API. <https://nominatim.openstreetmap.org/>, 2024.
- [8] **PHP Group.** PHP 8.2 Manual. <https://www.php.net/manual/en/>, 2024.
- [9] **MySQL.** MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>, 2024.
- [10] **Composer Project.** Composer — PHP Dependency Manager. <https://getcomposer.org/doc/>, 2024.
- [11] **Stauffer, M..** Laravel: Up and Running (3rd ed.). O'Reilly Media, 2023.
- [12] **OWASP Foundation.** OWASP Top Ten Security Risks. <https://owasp.org/www-project-top-ten/>, 2024.
- [13] **Bootstrap Team.** Bootstrap Icons v1.10. <https://icons.getbootstrap.com/>, 2024.
- [14] **Vite.** Vite Documentation. <https://vitejs.dev/>, 2024.